

Best Practices for Machine Readable Data and Code Sharing in Psychology

Daniël Lakens¹, Cristian Mesquida², and Lisa DeBruine³

¹Industrial Engineering and Innovation Sciences, Eindhoven University of Technology

²Industrial Engineering and Innovation Sciences, Eindhoven University of Technology

³Institute of Neuroscience and Psychology, University of Glasgow

Author Note

Daniël Lakens  <https://orcid.org/0000-0002-8393-5316>

Cristian Mesquida  <https://orcid.org/0000-0002-1542-8355>

Lisa DeBruine  <https://orcid.org/0000-0002-7523-5539>

A reproducible version of this manuscript available at on GitHub (https://github.com/scienceverse/machine_readable_FAIR_data_and_code). The authors have no conflict of interest to declare. This work was funded with the support of the Netherlands Organization for Scientific Research (NWO Vici grant VI.C.241.013). Author roles were classified using the Contributor Role Taxonomy (CRediT; <https://credit.niso.org/>) as follows: Daniël Lakens: conceptualization, writing, funding acquisition; Cristian Mesquida: conceptualization, writing; Lisa DeBruine: conceptualization, writing

Correspondence concerning this article should be addressed to Daniël Lakens, Industrial Engineering and Innovation Sciences, Eindhoven University of Technology, Den Dolech 2, 5600 MB, Eindhoven, Email: D.Lakens@tue.nl

Abstract

The FAIR principles state that research materials should be Findable, Accessible, Interoperable, and Reusable ([Wilkinson et al., 2016](#)). Best practices for FAIR data and code involve the creation of consistently structured repositories and standardised, machine-readable meta-data, but adhering to these best practices is a complex and effortful task. Here we describe a variety of best practices and existing standards, and introduce software that can facilitate these practices.

Keywords: FAIR, Data sharing, Reproducibility, Open Science

Best Practices for Machine Readable Data and Code Sharing in Psychology

Access to the data and code that underlie findings in the scientific literature allows peers to scrutinize the evidential basis for empirical claims, and to increase the efficiency of knowledge generation through re-use. The [European Code of Conduct for Research Integrity](#) states: “Researchers, research institutions, and organisations ensure that access to data is as open as possible, as closed as necessary, and where appropriate in line with the FAIR Principles (Findable, Accessible, Interoperable and Reusable) for data management.” Similarly, the [Dutch Code of Conduct for Research Integrity](#) states that: “Ensure that, in accordance with the FAIR principles, data is open and accessible to the extent possible and remains confidential to the extent necessary.” Accordingly, research materials should be managed in accordance with the FAIR Principles ([Wilkinson et al., 2016](#)).

The FAIR Principles were explicitly developed with both human and machine users in mind. Although it is important that individual can find, access, and re-use data, the benefits of FAIR data and code can only be realised when scientific resources are also accessible and processable by machines. Therefore, research data and code should be shared in machine-readable formats (e.g., see [Lakens & DeBruine, 2021](#) for an example of the benefits of machine-readable hypothesis). Machine-readable FAIR data and code will make it possible to retrieve and integrate data into large, searchable databases, and facilitate the automation of tasks, such as checking data and code for adherence to best research practices. This guide summarises best practices for sharing data and code in ways that make them findable, accessible, interoperable, and reusable by both humans and machines.

A machine can only find data and code reliably and automatically if they have a stable digital identifier and metadata; can be accessed through standardized protocols (e.g., HTTPS or an Application Programming Interface (API)); are provided in formats that can be used by other software (e.g., non-proprietary formats accompanied by metadata describing how files should be read in); and are re-usable (e.g., by providing a codebook and a license).

When data and code are shared this way, an automated computational system can identify what files contain — including both their format and purpose — based on metadata or filenames; determine whether they can be accessed and reused, for example given the license

or privacy regulations; and process the data and code accordingly. Ideally, each of these steps should be unambiguous and deterministic, minimizing the need for probabilistic inference by machine-learning algorithms or large language models. This guide is intended for researchers and data managers. It discusses best research practices for making data and code machine-readable and FAIR, including where to share data and code, repository organization, file naming, file formats, codebooks, README and requirement files, data layout, column naming, codebook generation, specification of scales and measures and adherence to existing schemas and standards.

Practices

1 Repository Choice

There are many data repositories, each with different strengths and limitations. When choosing a data repository, consider whether it allows you to share anonymized versions of the repository and specific files, or control who can access your data. You should also consider whether the repository provides persistent identifiers (e.g., DOIs), stable and unchangeable versions, version control and appropriate metadata. Other considerations include compatibility with regulations such as the General Data Protection Regulation (GDPR), funder requirements, and any limitations on the types or quantities of data that can be archived.

Some repositories commonly used in psychology include Zenodo, GitHub, the Open Science Framework, ResearchBox, Figshare, and PsyArchives. In addition, countries and universities may provide dedicated research data repositories, such as the UK Data Service or DataverseNL. GitHub is primarily a platform for developing and collaborating on code rather than a data repository and does not provide stable identifiers. However, published releases can be archived through Zenodo, which assigns a DOI and preserves specific versions of a repository. Once a project is published on Zenodo, its files and persistent identifier cannot be modified, and subsequent changes are represented as new versions.

The suitability of a repository also depends on whether it supports machine access. For example, repositories that provide public APIs allow automated systems to retrieve metadata and files without relying on manual interaction with a web interface. For example, ResearchBox and PsychArchives currently do not have a public API, which makes it difficult

for a machine to access files. Similarly, repositories that support anonymous or restricted-access sharing can be particularly useful during peer review, which can easily be done on PsychArchives, Dataverse, ResearchBox, and the Open Science Framework, and with some effort on Zenodo and GitHub. For sharing anonymised data and research materials, Zenodo is often a suitable choice. It provides persistent DOIs, versioning and options for restricting access to files when necessary, including for materials under review or data that cannot be made publicly available. However, Zenodo should not be used to share identifiable or sensitive personal data simply because it can be restricted. Zenodo is hosted by CERN, an intergovernmental organization that is not subject to the European General Data Protection Regulations, which makes it a less appropriate choice for pseudo-anonymized data.

When choosing a data repository, therefore, do not simply ask whether it allows you to upload your files. Consider whether its infrastructure support the level of openness, persistence, accessibility, and machine readability required for your research materials to adhere to FAIR principles.

2 Which Files to Share

Data repositories might contain files that researchers share to facilitate *replication* such as study materials, stimuli, questionnaires, and experimental procedures, as well as files that facilitate *reuse*, such as data and code to reproduce results or conduct new analyses. Both types of files should be shared, but they serve different purposes and may require different purposes to sharing. Study materials vary considerably across studies and are rarely processed by a machine. Data and code, in contrast, contain structured information that other researchers and computational systems might want to access and process directly - for example, to enable living meta-analyses, or to search for recent studies that have used a particular questionnaire.

Where possible, researchers should share raw data, meaning the data in the form in which it were originally generated or collected. Examples include exported Qualtrics files, audio recordings, or individual-participant EEG data. Raw data provide an important record of the original observations and allow subsequent users to understand, and reproduce the data processing steps that were applied to the raw data.

Sharing raw data is not always possible or appropriate. For example, the raw data

might not have been collected in a digital format, may contain information that cannot be adequately anonymised, or may be subject to ethical or legal restrictions. In such cases, researchers should share the earliest digital representation that can be made available, such as scanned surveys, or anonymized transcripts. When data require processing before they can be analysed or shared — for example, when personally identifying columns must be removed or unstructured data must be converted to a tabular format — researchers should use code to perform these transformations whenever possible. The resulting minimally processed data should be shared along the code to them. The README file should explain how the processed data was created from the raw data and identify any transformations that were applied.

Using code rather than manually editing the raw data has two important advantages. First, it creates an auditable record of how the data were transformed, allowing others to inspect the processing workflow for potential errors. Second, it makes the transformation reproducible: the same code can be rerun if errors are identified, the processing needs to be updated, or a new version of the data needs to be generated. Manual alterations to the raw data should therefore be avoided, and any unavoidable manual processing should be otherwise documented.

In addition to data, and code, researchers may want to share output files, including the results of statistical analyses, tables, and figures. Ideally, these output files are computationally reproducible from the shared data and code. Nevertheless, it can be useful to share outputs even when they are fully reproducible, particularly when reproducing them requires substantial computational time or resources, such as the results of simulations. Researchers may also want to share documentation, including codebooks, preregistration documents, the manuscript, supplementary materials, and metadata files such as README files. These files provide important context for interpreting and reusing the data and code and should be clearly identified in the repository.

Online data repositories often provide generous storage limits. For example, Zenodo currently allows up to 50GB per record, with a maximum of 100 files. Large files can create challenges for (automated) reuse. Users may not have sufficient disk space or time to download files that are not relevant to their intended use. Downloading and unzipping a 10 GB

archive, for example, requires substantially more resources than downloading 1 MB file. Repositories should therefore be kept as small as possible, and files should be organised and labelled so that they can be downloaded selectively. This is particularly important for automated tools, which may impose limits on the size of files or repositories they are able to process. When a repository size exceeds these limits, an automated tool may need to prioritize certain files, download only a subset of the data, or retrieve only certain files. For example, an automated tool intended to reproduce analyses reported in a manuscript may need to download the analysis dataset and code while ignoring large stimulus files such as videos.

Compression can substantially reduce the size of large data files. For large CSV files, gzip (.gz) compression is particularly useful because compressed CSV files can still be read directly without first being decompressed. We recommend using gzip compression for CSV files larger than approximately 10 MB to substantially reduce the size of the repository. For collection of multiple files, a .zip archive can be useful. ZIP archives contains the end central directory that records information about the files they contain, including the name and size of each file. Because this directory is located near the end of the archive (within the last 128 KB), software can use HTTP range retrieve it without downloading the entire archive. This makes it possible to determine whether a .zip archive contains files that are relevant to their intended use before downloading the archive in full. ZIP should therefore be preferred as archive format. Proprietary archive formats such as .rar should be avoided because they are less suitable for interoperable use.

Clearly label files to facilitate selective downloads. If a user wants to reproduce the data analysis, and a repository contains 14 GB of video files used as stimuli, they should be able to download only the data and code files needed for the analysis. This prevents the waste of time, computational resources and storage. Software that automatically downloads the files from research repositories will similarly need to impose limits on the size of individual files and entire repositories. For example, a tool that allows peer reviewers to automatically check whether reported results can be reproduced from the shared data and code may only be able to download files that fit within the available storage the peer reviewer has available. The practice of clearly labeling files in repositories is also important to distinguish links to repositories that

contain existing software used in research (e.g., a GitHub link to a software tool created by others that was used to process physiological data) from links to repositories that contain code generated by the researchers themselves for their research (e.g., code for an agent-based model that researchers have created as part of their research).

Some files can be completely reproduced from other files in a repository. For example, code for simulations might generate a large CSV file containing the results of hundreds of thousands of simulations. Labeling such files as ‘output’ will make it possible to download them only when needed. Because long-term storage of output files comes at cost, outputs should generally only be stored when they substantially reduce computational burden of reproduction or facilitate verification of the reported results. Computational reproducibility refers to the ability to produce the same outputs from the same data and code inputs (?). Computational reproducibility is often threatened by different versions of software, packages or computational settings. To increase computational reproducibility, the software and dependencies required to run the analysis should be clearly specified. For example, R projects can include an `renv.lock` file and Python projects a `requirements.txt` or `environment.yml` file. For analyses that depend on system libraries or software beyond R or Python packages, containerized solutions such as Docker, Nix, or Code Ocean provide stronger guarantees that the computational environment can be reconstructed in the future.

3 File and Folder Naming Conventions

As unanimous agreement on naming conventions is difficult to achieve, folder structure and naming conventions of folders and files will vary between research projects and fields. Nevertheless, some basic principles should be applied to facilitate machine-readable repositories and automated processing.

Folders are used to group related files. For scientific papers, folders can represent projects, such as single studies, and categories of files. Most research files can be organised into the folders data, code, outputs, materials, and documentation, with an ‘other’ folder for files that do not fit the other five categories. The top-level folder is associated with a paper, and it contains files that apply to all sub-folders in the repository. An example of a file that belongs to the top level folder is a README – a text document that provides a human-readable

description of the repository contents and instructions on reusing them.

The Psych-DS format, a data structure for research in the social sciences, recommends that repositories should contain a subfolder for each project in which data were collected. For multi-study manuscripts, this means that the repository contains a subfolder for each study (e.g., ‘pilot-classification’, ‘study-mood-classification’, ‘study-moderator-effect-mood’). Studies should be named consistently and clearly from the first documentation such as an ethical review application or preregistration. Because studies may not be communicated in the same order in which they were conducted, and because numbers do not describe what a study is about, avoid refer to studies simply as ‘study-1’, ‘study-2’, etc. Instead, use short, descriptive labels that assist machines to link data in a repository to a specific study in a manuscript. Once the final order of studies has been determined (e.g., when writing the manuscript), a number can be added to each study subfolder (e.g., ‘01-pilot-classification’, ‘02-study-mood-classification’).

At a minimum, each study folder should contain a ‘data’ folder. Data files should be accompanied by a codebook. Some analysis software, such as JASP and Jamovi, stores data, codebook information, analyses, and output in a single file (i.e., a .jasp or .omv file). This removes the need for additional files and folders: a single .jasp or .omv file in the data folder will allow a machine to retrieve all the information it needs to reproduce the analyses. If all analyses can be performed in JASP or Jamovi, the easiest solution for machine-readable FAIR data and code sharing is therefore to import the data, enter the codebook information, perform the analyses, and share the resulting .jasp or .omv file. This may often be an easier and more reproducible than performing the analyses in R, where making data and code equally FAIR requires substantially more work and expertise.

If a standalone codebook is needed, software packages that can assist with its creation ([Arslan, 2019](#)). When data are analyzed using individual code files, these should be placed in a ‘code’ folder. A study folder might also contain an ‘output’ folder for results generated by analysis scripts. Repositories may additionally contain a ‘documentation’ folder for documents such as the preregistration document and manuscript, and a ‘materials’ folder for stimuli, survey files, and other study materials.

Software will likely search for these folders using a range of aliases. For example, a documentation folder might be called ‘reports’ or ‘docs’, the materials folder might be called ‘assets’ or ‘stimuli’. As long as the variation in filenames is limited, software can search for a predefined list of names to identify folders containing particular types of files. This reduces reliance on large language models for file classification, which is less reliable and more computationally expensive than rule-based classification.

Files and folders should be named so that they are both human-readable, and facilitates classification by machines. Because files often need to be classified according to multiple properties (e.g., a unique identifier, and the type of file), a machine should be able to distinguish between these properties in a filename. A useful convention is to use underscores to distinguish different properties. For example, the filename ‘study-01-classification_data.csv’ allows a machine to identify both the unique identifier (i.e., study-01) and the file type (i.e., data). For classification purposes, the use of underscores or dashes is less important than consistently including terms that indicate file’s category. Even a CamelCase filename such as ‘codePowerSimulationStudy1’ could be classified by a machine if it contains an informative category term.

What is essential is that each filename contains a word that allows a machine to classify it into a category. For scientific data, the main categories are data, code, output, documentation, and materials. Filenames are an important source of information for machines attempting to classify the contents of a repository, and should therefore include one of these five words or a sufficiently similar word, such as `data_dictionary`. `stimuli`). Filenames should also include a descriptive keyword indicating their specific content (e.g., ‘preprint_documentation.pdf’). Given the lack of unanimous agreement on file naming conventions, machines may need to classify files using filenames and extension rules, with a large-language-model as a fallback for files that remain ambiguous after rule-based classification.

Some software might interpret names with and without capitalisation as different files (e.g., `Data.csv` versus `data.csv`). Be consistent in the use of capitals. In general, it is recommended to only use lowercase letters, unless schemas and other data standards require

capitalized filenames. Avoid spaces and special characters in filenames. Filenames should also allow for lexicographic sorting of numbered files. For example, if ‘2-data’ precedes ‘11-data’ name the files ‘02-data’ and ‘11-data’. If you plan to limit the number of files in a repository to 99 (as required by Zenodo), two digits should be sufficient. However, studies with a large number of stimuli or individual data files per participant, might require more digits for the files contained within a .zip archive. Dates should be provided in YYYYMMDD format, following the ISO 8601 basic calendar format (‘20260724_data.csv’), although the extended format (‘2026-07-24_data.csv’) may also be used when required by a particular convention.

The use of subfolders can lead to long file and folder names. Certain file systems and software tools limit filenames to 255 characters. Furthermore, when a repository is downloaded, the local folder path is added to the path of every file. For example, if a repository is downloaded to C:\Users\dlakens\Downloads, the filenames in the repository should remain below 228 characters. As a practical recommendation, try to keep the directory names (e.g., ‘study-1-representative-sample/individual-participant-files/accuracy/’) below 100 characters and filenames below 50 characters, leaving around 100 characters for the local folder path.

Names should match the terminology used in the manuscript. If the manuscript refers to ‘Study 1’, for example, the corresponding study folder should use the same identifier (‘Study 1’) rather than a different label such as ‘Survey 1’ or ‘Aggression Study’. This allows machines to link data and code in multi-study papers to the appropriate section of the manuscript. Meta-data (for example following the RO-Crate standard, as discussed in the next section) can provide additional information about files and folders that cannot be reliably encoded in their names alone.

4 Describe the Content and Organization of the Files

A common recommendation for data and code sharing is to add a README document to every repository. README documents are useful for re-users who manually browse through a repository, and should therefore be included to describe its contents and provide instructions for reuse. However, a README is not very useful for machine processing. README documents should therefore be complemented by a structured counterpart

containing information that cannot easily be retrieved from the files in the repository and that is of general interest for re-users.

This structured metadata file should provide information about the repository and its files that is difficult to retrieve from the files themselves. It should also identify the research project to which the files belong to and who created it. While researchers might assume that re-users will find their files through the article they have published, machines are more likely to find repositories directly. Therefore, the meta-data should describe the project independently of the manuscript and link to the manuscript DOI when one exists. Although good file naming should allow re-users to understand the contents of the files in a repository, adding this information automatically to the JSON file also provides a means of validating that filenames were unambiguous and provides ground truth when names may be necessarily ambiguous.

Adding meta-data to a repository is complex because multiple standards operate at different levels. Dublin Core metadata standards store generic information about a repository, such as its title, description, creator, and license. The [DDI Alliance](#) provides ISO-standard specifications designed to describe research data in the social, demographic, economic, and health sciences. The [DDI-Lifecycle 3.3](#) standard can be used to describe study design, sampling, and data, and supports the reuse of data. Field-specific meta-data standard can be developed to describe files common in specific research areas, such as [BIDS for brain data](#), or [Psych-DS](#) for research data in psychology. Finally, there are also metadata standards for specific types of research data within specific fields, such as the [Open Scales Database](#) specification for scales in psychology.

There is a clear need for field-specific guidelines on which meta-data should be shared. Tools should be developed to make it easy for researchers to generate JSON structured metadata files that comply with widely used metadata specifications. These recommendations will likely drawn on existing standards, such as the [DDI-Lifecycle 3.3](#) for the study, sample, and data-handling descriptions, [CRediT](#) for contributor roles, [schema.org](#) for bibliographic metadata, and the RO-crate specification (?) to document the relationship between files in a repository and support machine readable FAIR principles.

Some information is difficult to retrieve from a paper but valuable for reuse. Such

information can be directly encoded into metadata files. Which information is important enough to be included is a matter on which research fields should reach consensus. In many empirical disciplines, information about the sample size is worth documenting because the amount of information collected is relevant for reuse in meta-analyses and partly determines the strength of the evidence. The DDI-Lifecycle schema provides standardized metadata for sample eligibility criteria, inclusion and exclusion criteria, recruitment, the target sample size, the number of completed responses, and the resulting response rate.

As data provenance – a log of how a study’s data came into existence – becomes increasingly important, metadata could also describe the time window during which data was collected, the situation in which it was collected (for example, whether sessions were supervised, individual or group, or ran during an exam period), and the mode of collection (e.g., an online survey, in person interviews). DDI’s `LifecycleEvent` schema allows researchers to list each study’s data-handling steps as a series of events, stating what happened, when, and who was responsible. This makes it possible to log anyone who handled the data but is not an author. This makes it possible to document people who handled the data but not authors. These could include a research assistant who ran the sessions, a technician who exported the files, or a data manager who curated and archived the files. Recording these contributions makes this labour visible and provides a clear record of responsibility for each data processing step, which may be particularly important when research integrity concerns arise. As a final example, the increased interest in assessing whether manuscripts adhere to open science practices could motivate the development of a dedicated metadata schema for open science practices. Such schema could indicate whether all data that can be shared have been made available, whether the code needed to reproduce all reported results has been shared, whether the materials needed to replicate a study have been made available, and whether the study has been preregistered in a study registry.

In addition to the general metadata discussed above, individual studies can be represented as Psych-DS compliant subfolders to facilitate data reuse. The [Psych-DS](#) standard requires a `dataset_description.json` file, which provides a machine-readable description of a specific dataset, including every variable, its measurement scale, its value range, the

originating data file, and potentially additional information such as descriptive statistics. Unlike `ro-crate-metadata.json`, which describes provenance, contributions, and general properties of the data and code, this file describes what the data mean, enabling software to more accurately process variables in data files.

Machine-readable data descriptions support findability, reuse and reproducible analyses. They also allow users to search databases for studies that collected specific data. The Psych-DS JSON-LD document is based on Schema.org's Dataset model, extended with Psych-DS specific fields. These JSON files can be generated automatically from well-organized repositories, if they contain an accurate codebook, and use sensible file and folder names.

Even more fine-grained schemas exist for subsets of data files, such as psychological scales. The [Open Scale Definition](#) (OSD) is a machine-readable format for sharing metadata about questionnaires and rating instruments. It is part of the Psychology Experiment Building Language, which was created for building and running psychological experiments ([Mueller & Piper, 2014](#)). An `.osd` file has two top-level parts: a `definition`, which describes the structure, in a language-neutral way and `translations`, which contains the actual text, keyed by language code, allowing the same structure to represent versions of a scale in different languages. The OSD specification requires `scale_info`, `likert_options`, `dimensions`, `items` and `scoring` (although an entry can be set to `NULL`). Scale information includes the name, code, abbreviation, citation to the original publication of the scale, license information, and other descriptive information. The JSON file can also contain information about the Likert-scale response options, the dimensions of the scale (such as subscales, and which items belong to which subscale), how the scale is scored, and the specific text of each item in different translations when available. FAIR scale metadata facilitates reuse and can also promote standardization in how scales are used and scored across studies.

5 File Formats

Data should be stored in open, machine-readable formats, such as `.csv`, or in well-supported software-specific files such as `.svp`, `.jasp` or `.omv`. Very large data files should be stored in file formats that allow partial downloads of information through range requests.

For example, CSV files allow parts of the text to be retrieved without downloading the entire file, whereas this may not be possible for other other formats such as .xlsx.

When data are collected with commercial software, or when data are not collected digitally (e.g., a questionnaire printed on paper), it is may not be possible to make raw data (i.e., data in the form they were originally observed) available in open format. In such cases, the raw data files should be stored in a folder called “raw” inside the data folder, preferably in a .zip archive if there are many files or if the files are large. The raw data should be also exported, as directly as possible, to an open format such as csv. CSV files should follow the [RFC 4180 specification](#) and UTF-8 encoding. Commas used within a data value should be enclosed in double quotation marks (e.g., pp1, 34, "To be, or not to be."). Avoid using plain .txt files for data storage, as machines cannot reliably determine how to interpret their contents. Similarly, do not use .RData files to store R output when individual objects can be stored as .rds files. An .RData file can contain multiple objects, making it difficult for software to determine how what it contains. The file extension should accurately reflect the underlying format. For example, a JSON formatted file should be saved as .json, rather than as .csv or .txt.

Spreadsheets (.xls or .xlsx) can also be used to store data. The .xlsx format is an Open Office XML format that can be read in any open source-software packages. However, several practices should be avoided. First, one should not store separate datasets in multiple tabs. Because filenames are used to classify data files, a filename cannot distinguish the data contained in different tabs. Instead, store separate datasets as separate files. Second, data should be rectangular, with each column containing the same number of rows, so that they can be reliably imported into other software. Spreadsheets with merged cells cannot be read as a data frame, and spreadsheets with empty rows or columns can also cause problems.

Leave missing cells empty since software can replace empty cells with an empty value (e.g., as NA in R). Avoid coding scheme for missing values such as 999 = ‘no response’, 888 = ‘hardware failure’. If such practice cannot be avoided, for example because you do not have control over the raw data, document the coding scheme in the metadata or codebook. Do not rely on colour coding, comments, or hyperlinks to communicate information. This information is often lost when the data are imported into other software. Instead, record it in dedicated

columns (e.g., a ‘notes’ column). CSV files and spreadsheets should contain headers in the first row, with a single row used to label the columns.

6 Codebooks and data dictionaries

A codebook, or data dictionary, describes the contents of data files. The two terms are often used interchangeably and largely contain the same information. Buchanan et al. (2021) note that a codebook customarily describes survey variables and can be created during data collection, whereas a data dictionary is more commonly created in the planning stage, and may additionally document the structure of data files. Regardless of terminology, every data file (or set of data files with identical structure) should have a codebook. Data files should have a single-row header containing column names, and the codebook should explain what these names represent and what the values in the columns mean. Some data formats (e.g., SPSS, Stata, SAS, JASP, jamovi, Qualtrics) support variable and value labels, and can therefore contain an embedded codebook.

A complete codebook should contain six types of information. First, it should contain the column name exactly as it appears in the data file (e.g., `panas-1`). Second, it should contain a label describing the column in prose (e.g., `positive and negative affect schedule item 1`). If the data did not originate from a survey question, the label should describe what the values in the column represent (e.g., reaction times, correct or false responses, reading times, etc). The goal of the label is to provide a human-readable explanation of the variable, and add context. If the item is part of an existing scale, the full name of the scale should also be provided here. At a minimum, a codebook needs a column name and a label.

Third, for data with a limited number of response options, the codebook should provide value labels. For scales, these describe how the response options were labeled (e.g., `1 = very negative; 2 = somewhat negative; 3 = neutral; 4 = somewhat positive; 5 = very positive`). If only the endpoints are labeled, then only enter those two values (e.g., `1 = very negative; 5 = very positive`). Fourth, for data that was generated based on survey questions, enter the verbatim question text (Interested). Fourth, for data generated from survey questions, the verbatim question or item text presented to participants should be

provided. Fifth, if it is not possible to specify a single coding scheme for missing data across all data columns in the repository metadata (see above), an individual missing-value coding scheme can be provided for each column. Sixth, coding instructions can be provided when useful. One important coding instruction is `reverse scored` for items that should be reverse coded before combining into a scale mean. Other coding instructions can describe how a variable was or will be used in the calculation of the scale (e.g., `mean of PANAS items 1–10`) or whether scores need to be transformed before analysis. If coding instructions are too complex to represent in a tabular format, refer to a README or other document that explains the coding or, ideally, provide code that transforms the data into scores.

The easiest way to create a codebook is often to use data analysis software, and complete the relevant codebook fields within the software. If your data are in SPSS (`.sav`), Stata (`.dta`), SAS (`.sas7bdat`), JASP (`.jasp`), jamovi (`.omv`), or Qualtrics (`.qsfx`), variable and value labels can be entered in the software and shared as part of the data file. An important benefit of these formats is that they combine the data and codebook information in a single file. However, these formats do not necessarily contain all the information needed for a complete codebook. Beyond `.sav` and `.jasp` files, the other formats do not allow users to specify missing values. Beyond Qualtrics, these formats do not allow specification of the exact survey item, and none provides a dedicated field for coding instructions. Therefore, if any of these additional fields are needed, a separate codebook should be provided.

PsychDS provides a schema for describing data files in a JSON-formatted codebook. However, many codebooks have a tabular structure, with column headers specifying the type of information recorded for each variable. An example is provided in Table 1.

Although tabular codebooks can be created as `.csv` files, it may be most convenient to use a spreadsheet. When creating the codebook, the column names can be copied from the data file, and transposed (i.e., converted from one row into one column) before being pasted in the spreadsheet. This helps ensure that the column names in the data file and codebook match exactly. Make sure values are stored as text, as Excel might transform values such as 1–5 into a date such as 5-Jan.

The codebook can be stored as an `.ods`, `.xlsx`, or `.csv` file. The first two formats are

Table 1
: *Example of a codebook*

variable-name	label	values	missing-values	question	coding-instructions
pid	Participant identifier				
age	Age in years		-99	What is your age in years?	
gender	Self-reported gender	1 = Man; 2 = Woman; 3 = Non-binary	99	Which best describes your gender?	
bds_1	Breakup Distress item 1	1 (Strongly disagree) to 5 (Strongly agree)	99	I feel overwhelmed by the breakup.	
bds_3	Breakup Distress item 3	1 (Strongly disagree) to 5 (Strongly agree)	99	I am coping well.	reverse scored: 6 - bds_3
bds_mean	Breakup Distress mean score				mean of bds_1 to bds_10 after reversing bds_3

easier to read by humans, while CSV files are more interoperable. Do not store codebooks as Word documents or PDF files, as information in these formats cannot be reliably parsed by machines. Name the file codebook or data-dictionary. If there is only one data file in the same folder, there is no need to include the data filename in the codebook, as the folder establishes the relationship between the files. If a folder contains multiple data files, however, use the filename to match each codebook to its corresponding data file (e.g., `pilot-1a_data.csv` and `pilot-1a_codebook.csv`).

7 Tabular data files

Data files should be rectangular tables with a tidy structure: one header row, one variable per column, and one observation per row. Analysis software such as SPSS, JASP and Jamovi will require this structure, but when data files are created or edited in spreadsheets, it is very tempting to perform actions that deviate from this standard. Common problems include headers spanning multiple rows, empty columns or rows, and notes written below or

alongside the data.

Best practices for column names are more restrictive than best practices for filenames. Column names should contain only letters, digits, and underscores (e.g., `participant_id`, `mean_reaction_time`) should be used to make them easier to reference in analysis code. In R, a column can then be referenced as `df$participant_id`. If a column name contains spaces or dashes, it needs to be quoted (e.g., `df$`participant-id`` or `df$`participant id``). Special characters often need to be escaped to print, or may be automatically renamed when a data file is imported into other software. This can create problems, including identical column names when two columns only differ in a special character. The same applies to characters with diacritics. Keep column names shorter than 32 characters, as this is the limit for some analysis software packages. Column names should be unique within a data file, and columns representing the same information across different files should use the same name (e.g., `participant_id`). Column names must also match the corresponding names in the codebook exactly.

Use descriptive column names that can be understood without consulting the analysis code (e.g., use `self_esteem_scale_1` rather than `SE1`, and never use generic names such as `VAR1`). Where possible, use terminology that matches how variables are described in the manuscript. When columns belong to the same scale or construct, use the same prefix followed by an increasing number (e.g., `self_esteem_scale_1`, `self_esteem_scale_2`, `self_esteem_scale_3`). If a scale contains subscales, the same convention can be extended to identify the subscale (e.g., `panas_1_positive_1` for the first item of the positive affect subscale of the Positive and Negative Affect Schedule). This makes it easier for machines to identify scales and subscales automatically. Scale items belonging to the same scale should be placed in adjacent columns where possible. When multiple columns relate to the same response, such as the response value and its response time, clearly indicate the property represented by each column (e.g., `mood_1_response`, `mood_2_response`, `mood_1_reactiontime`, `mood_2_reactiontime`).

There should be no columns or rows containing exclusively empty values. Blank cells are ambiguous: they may indicate that a value was not recorded, that a column was deleted or

that a column was accidentally left empty. If a column contains no recorded values, replace the empty cells during data preprocessing with a value that indicates nothing was recorded (e.g., ‘none’ or 0) and document the meaning of this value in the codebook. Column names and values should not contain leading or trailing whitespace (e.g., use yes, not yes or yes). Avoid mixing text and numeric values in columns that are otherwise numeric. For example, adding a single text value such as ‘no response’ to a column of numerical responses can cause software to interpret the entire column as text rather than numeric. Instead, use a consistent missing-value code and document it in the codebook. Before sharing data publicly, carefully check whether the raw data contains personal information such email addresses or IP addresses, and remove or anonymize such information as appropriate. Open-text responses should also be screened for identifying information before the data are shared.

Discussion

Data and code should be shared where possible to facilitate verification, reproducibility and reuse. Repositories containing FAIR data and code should allow machines to find and access all content. This requires data and code to be made available in online repositories that can feasibly be downloaded; files and folders to be clearly named and organized in a consistent folder structure to allow automatic classification and identification of relationships between files; the repository contents to be described using machine-readable metadata; files to be shared in open and machine-readable formats; a codebook to be included included; data files to be well-structured; and, where relevant, metadata about measures and tasks to be provided. Schemas such as Psych-DS, RO-Crate JSON-LD, DDI-Lifecycle, CRediT, Open Scales Definition, and Behavense Data Model can facilitate FAIR data and code.

Adhering to best practices for FAIR data and code is complex and effortful. This document contains more than 100 recommendations and it is unreasonable to expect researchers or data managers to remember and implement them all. One solution to overcome the challenges of implementing all these recommendations and training of researcher is to automate the generation and validation of FAIR data and code as much as possible. Software could scan a repository and assist researchers in iteratively improving its FAIRness. Metacheck (<https://github.com/scienceverse/metacheck>) is one implementation of this approach: it scans a

repository, classifies files, retrieves information from data files and codebooks, and can assist in creating metadata files. Input from researchers remains essential for information that cannot be inferred automatically, but manual inputs should only be required where needed.

The potential of machine-readable data and code extends beyond individual repositories. If data repositories can be scanned automatically, information about their contents could be processed automatically, and stored in a central database. Researchers could then search for data files containing particular scales or tasks while filtering by characteristics of the sample such as demographic information. This could greatly facilitate the reuse of research data. Machine-readable code and data could also make automated reproducibility checks possible increasing the quality of scientific manuscripts, without increasing the burden on peer reviewers. More broadly, machine-readable repositories could help researchers demonstrate adherence to research-integrity requirements to FAIR data and code when possible. They could also allow universities, funders, and other research organizations to automatically check whether shared research outputs meet relevant data-sharing requirements.

As adding metadata to repositories remains challenging, and the number of best practices in data sharing makes it difficult to remember and implement them all, automated tools will therefore play a big role in the adoption of FAIR principles to data and code. If data and code can be processed by machines, it becomes possible not only to validate the extent to which a repository follows FAIR principles, but also to guide researchers interactively towards improving its findability, accessibility, interoperability, and reusability.

References

- Arslan, R. C. (2019). How to Automatically Document Data With the codebook Package to Facilitate Data Reuse. *Advances in Methods and Practices in Psychological Science*, 2(2), 169–187. <https://doi.org/10.1177/2515245919838783>
- Buchanan, E. M., Crain, S. E., Cunningham, A. L., Johnson, H. R., Stash, H., Papadatou-Pastou, M., Isager, P. M., Carlsson, R., & Aczel, B. (2021). Getting Started Creating Data Dictionaries: How to Create a Shareable Data Set. *Advances in Methods and Practices in Psychological Science*, 4(1), 2515245920928007. <https://doi.org/10.1177/2515245920928007>
- Lakens, D., & DeBruine, L. M. (2021). Improving Transparency, Falsifiability, and Rigor by Making Hypothesis Tests Machine-Readable. *Advances in Methods and Practices in Psychological Science*, 4(2), 2515245920970949. <https://doi.org/10.1177/2515245920970949>
- Mueller, S. T., & Piper, B. J. (2014). The Psychology Experiment Building Language (PEBL) and PEBL Test Battery. *Journal of Neuroscience Methods*, 222, 250–259. <https://doi.org/10.1016/j.jneumeth.2013.10.024>
- Wilkinson, M. D., Dumontier, M., Aalbersberg, Ij. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., Silva Santos, L. B. da, Bourne, P. E., et al. (2016). The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3(1), 160018.